

VAHAN SYSTEM OVERVIEW REPORT

Data collection, administration, and planned expansion

This report explains how the system collects vehicle registration data from VAHAN, stores and retrieves results, and supports daily operations. It also describes an expansion for monthly collection. Existing capabilities and proposed features are identified separately to support review and acceptance.

Overall architecture

In the current version, the administration interface creates and tracks jobs. The orchestration service assigns each job to one browser worker, receives its result, and stores it in the database. The worker interacts with VAHAN Public Report. Figure 1 shows the proposed shared queue and multi-worker architecture for scheduled monthly runs and immediate requests from the interface.

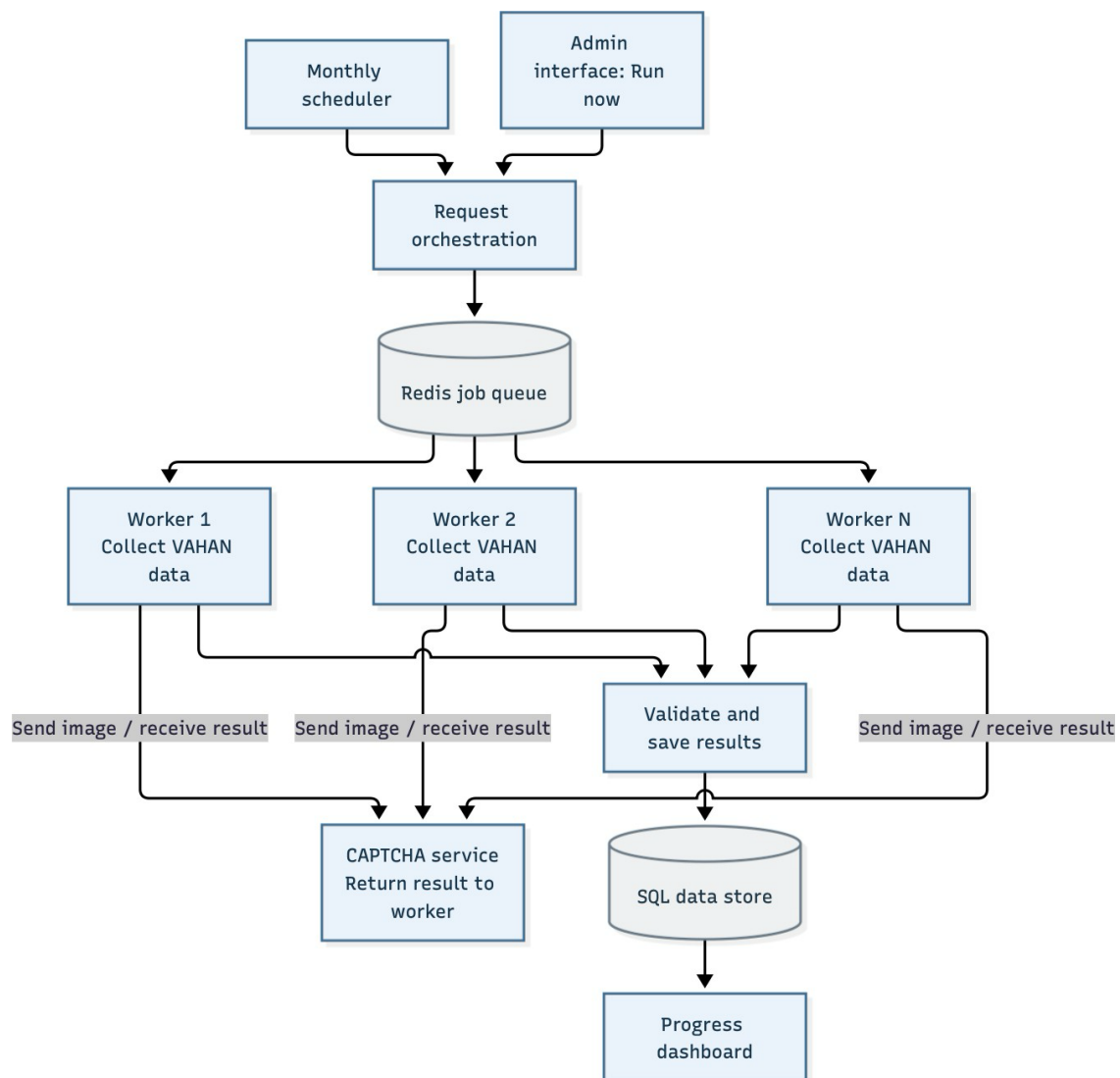


Figure 1. Proposed architecture: scheduled and immediate requests share a queue, and workers call a CAPTCHA service when verification is required.

Implementation status: the Redis queue, automatic monthly schedule, shared CAPTCHA service, and concurrent workers are proposed extensions. The current workflow uses one worker, coordinates batches from the interface, and offers local CAPTCHA recognition support with a manual entry option.

1. Data collection process

In the current workflow, the system builds a State–RTO list from VAHAN options, runs each case with the standard report filters, and records the result for each office. Figure 2 shows the data, no-data, and error outcomes.

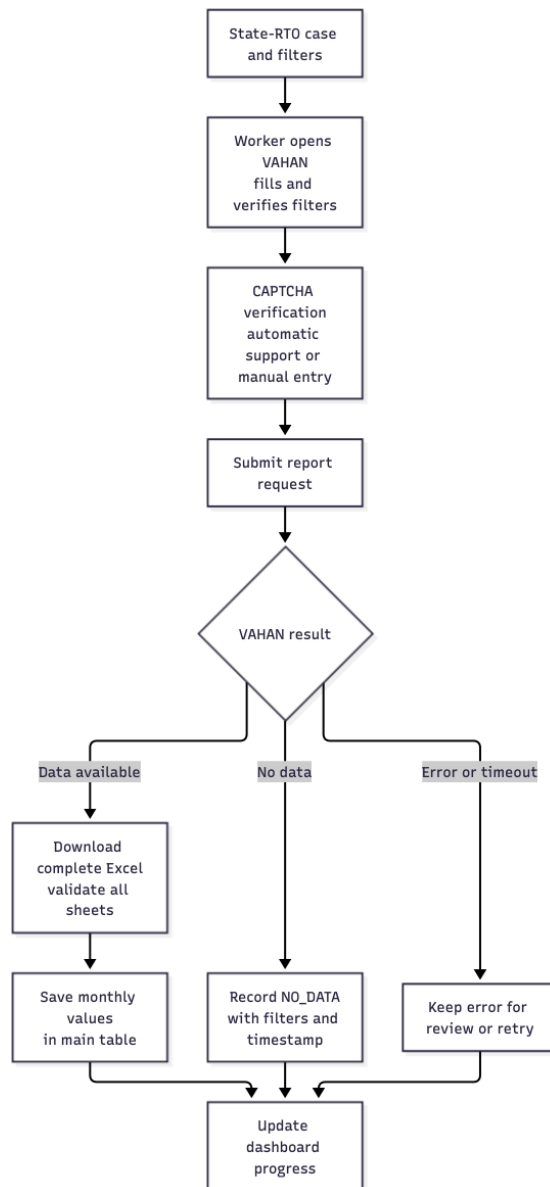


Figure 2. Current collection flow: a case is completed only after a valid result has been saved.

Step	How the system works	User outcome
Define scope	The system loads State/RTO options from VAHAN and builds cases for the current year with standard filters. The user can run all cases, start from one RTO, or run one RTO.	Each case has a defined scope, reducing the risk of collecting the wrong report.
Collect data	The worker opens VAHAN, fills the filters, handles verification, and waits for either a report or a confirmed no-data result.	The current case, its status, and elapsed time can be monitored.
Import data	When data is available, the worker downloads the complete Excel report. The system reads its sheets, validates context, and saves values by Maker, State, RTO, year, and month.	Data becomes available in the main table as soon as the save succeeds.
No data	A no-data outcome is recorded only when VAHAN displays a suitable confirmation.	An empty report remains distinct from a timeout or verification error.

The “OTHERS” group supplied by VAHAN is retained as OTHERS in the data. The system does not assign it to a specific manufacturer. A case is marked complete only after its data has been validated and saved successfully.

2. Administration interface functions

The interface brings together report creation, run monitoring and recovery, data lookup, account administration, and checks for changes to the VAHAN page. Figure 3 summarizes its three main areas.

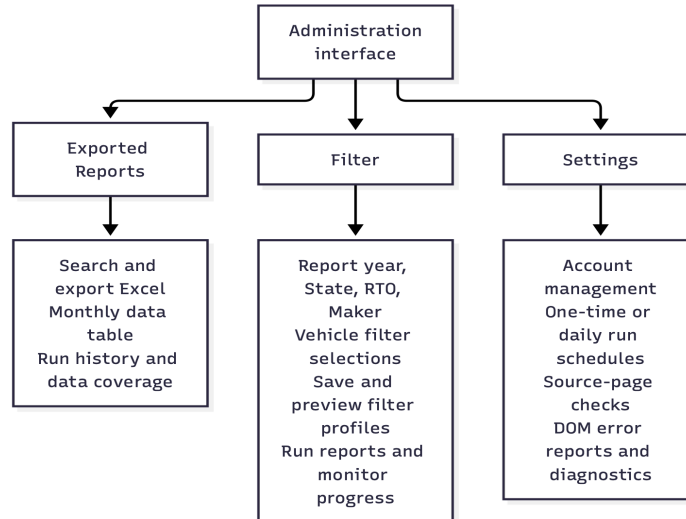


Figure 3. Functional map of the existing administration interface.

Function	Actions and information shown	User benefit
Configure filters	Select year, State, RTO, Maker, and vehicle filters; save profiles and preview cases.	Reuse a consistent reporting scope.
Create and run reports	Run all cases, one RTO, or start from a selected RTO; stop, continue, or restart.	Control report collection under one plan.
Monitor progress	Show the current case, result counts, Activity, and time per filter.	Track progress and cases needing attention.
Handle errors	Retry failures once after each 10-case group and the final partial group; manually retry remaining failures and copy errors.	Recover failures without rerunning successful cases.
Run history	Review, hide, or restore sessions while retaining saved data.	Trace and manage previous runs.
Exported reports	Search by year and State/RTO, view monthly data, and export matching results to Excel.	Reuse collected data.
Data coverage	Show confirmed RTO results and collect missing RTOs.	Fill gaps without repeating completed work.
Schedule report runs	Choose a saved profile, year, workers, and start time (UTC+07:00); run once or daily; view progress, enable, disable, stop, or delete schedules.	Run at planned times while services remain running.
Account administration	Manage sign-in, users, and role-based access.	Control access and responsibilities.
Source-page checks	Schedule checks, check immediately, and review results.	Detect VAHAN page changes early.
DOM error reporting	Show error codes, affected elements, expected/actual structure, and copyable diagnostics; block new work for incompatible changes.	Identify failures and support repairs.

2.1. Edit filter configurations and select cases

The standard State–RTO matrix currently uses fixed filters: the current calendar year, all supported Active and Archive statuses, the Two Wheeler category and its three subcategories, electric fuels, and Maker by Month Wise axes. Users can select the State/RTO run scope, but there is no screen for editing these standard filter values before the matrix is created.

The proposed extension would let an administrator edit and save a report configuration covering period and year, Active and Archive statuses, category and subcategory, fuel, emission, Maker, report axes, and State/RTO scope. The interface would preview the case list and let the user run all cases, start from a selected case, or run only that case. Values must be checked against VAHAN options before execution, and each run must retain the configuration used for later lookup and reconciliation.

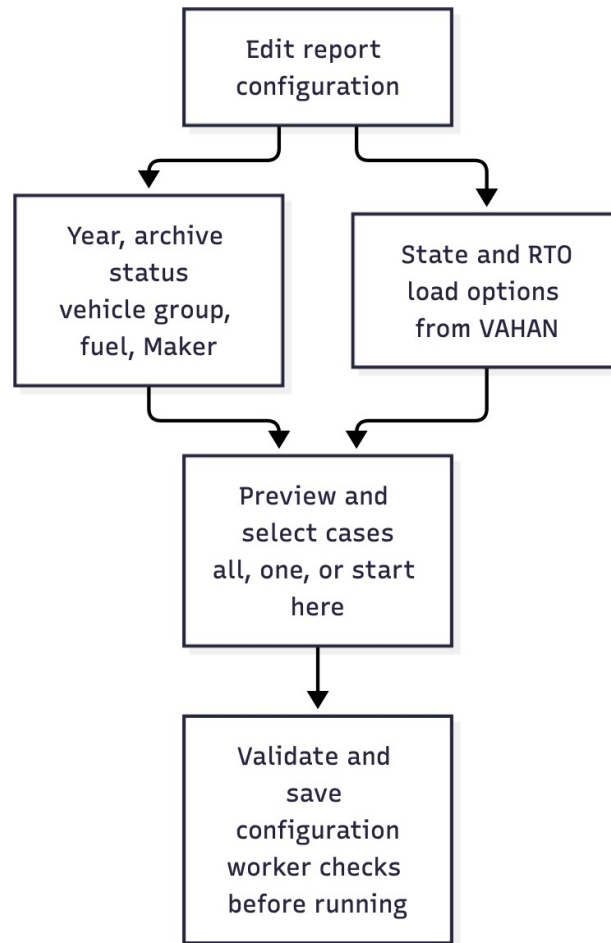


Figure 4. Proposed flow for editing filter configurations and selecting cases to run.

The filter-editing function in Figure 4 is a proposed extension, not an existing screen. Jobs already in progress continue with the filters saved at creation. Maker updates compare only data from the same filter scope and year.

3. Report data and reconciliation

The main report table stores 12 monthly values for each Maker, State, RTO, and report year. Job history records filters, timestamps, status, and errors. Update history shows whether data was added, already present, or requires review. All accounts use the same shared report dataset.

The system distinguishes an empty cell from zero, avoids duplicate imports of the same result, and reports completion only after a successful save. Missing or invalid source data is reported separately for operator review. A confirmed no-data result is also recorded separately from collection errors.

4. Operations, monitoring, and access control

A batch processes cases in order, revisits failed cases after every group of 10 reports, and saves progress so operators can continue from the correct position. Figure 5 shows the current retry policy.

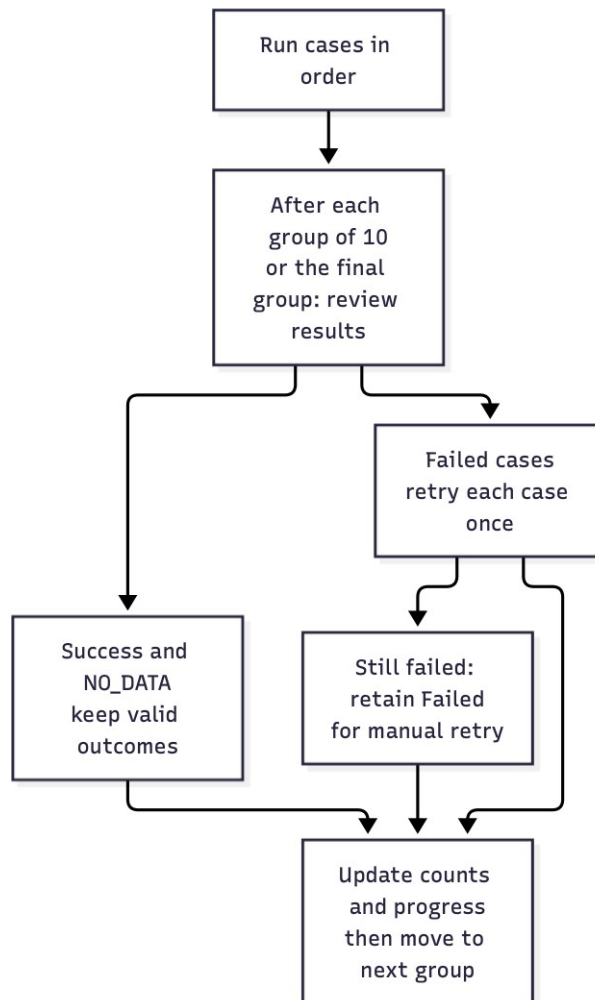


Figure 5. Batch retry and continuation flow; a valid no-data result is not treated as an error.

Area	How it is supported
Continue and retry	Batch progress is saved for continuation after a stop. Failed cases are retried within limits and remain visible if unresolved.
Monitoring	The dashboard shows worker and job status, recent activity, and VAHAN page-check results.
Authentication	Users sign in with individual accounts, and roles determine available administration functions.
Storage	Report values, run history, and update records are stored in the database for lookup, export, and reconciliation.

5. Planned expansion: monthly collection and multiple workers

Under the design in Figure 1, a monthly scheduler creates requests by State and reporting period. A user can also select a State and year/month in the interface and choose Run now. Both request sources use the same orchestrator, which checks for duplicates and places jobs in a queue for distribution to multiple workers.

Each worker receives one job, interacts with VAHAN, calls the CAPTCHA service when verification is required, and uses the result to continue. After the report has been checked and saved successfully, dashboard progress is updated. Failed jobs may be retried within defined limits, while unfinished jobs remain visible for follow-up.

This is a proposed design, not a feature accepted in the current release. Implementation requires agreed limits on VAHAN traffic, CAPTCHA-service capacity, duplicate prevention, and the criteria for confirming final data.

6. Delivery scope and evaluation criteria

The current release supports VAHAN reports with standard filters, State/RTO run selection, SQL storage, report lookup and export, run and account management, source-page checks, and selective Maker updates. An editable filter-configuration screen, monthly scheduling, a shared queue, and concurrent workers belong to the expansion described in this report.

To evaluate a collection run, compare the selected filters, counts of completed/with-data/no-data/failed cases, monthly values in the source report and saved table, Excel export, and the list of cases requiring follow-up. For a Maker update, verify the baseline, changed-Maker list, and the exact States and RTOs refreshed.

7. Database and data lifecycle

The current system uses PostgreSQL as the durable record for users, runs, source results, and update history. A browser worker can finish a case only after the API confirms that the result and its status have been committed.

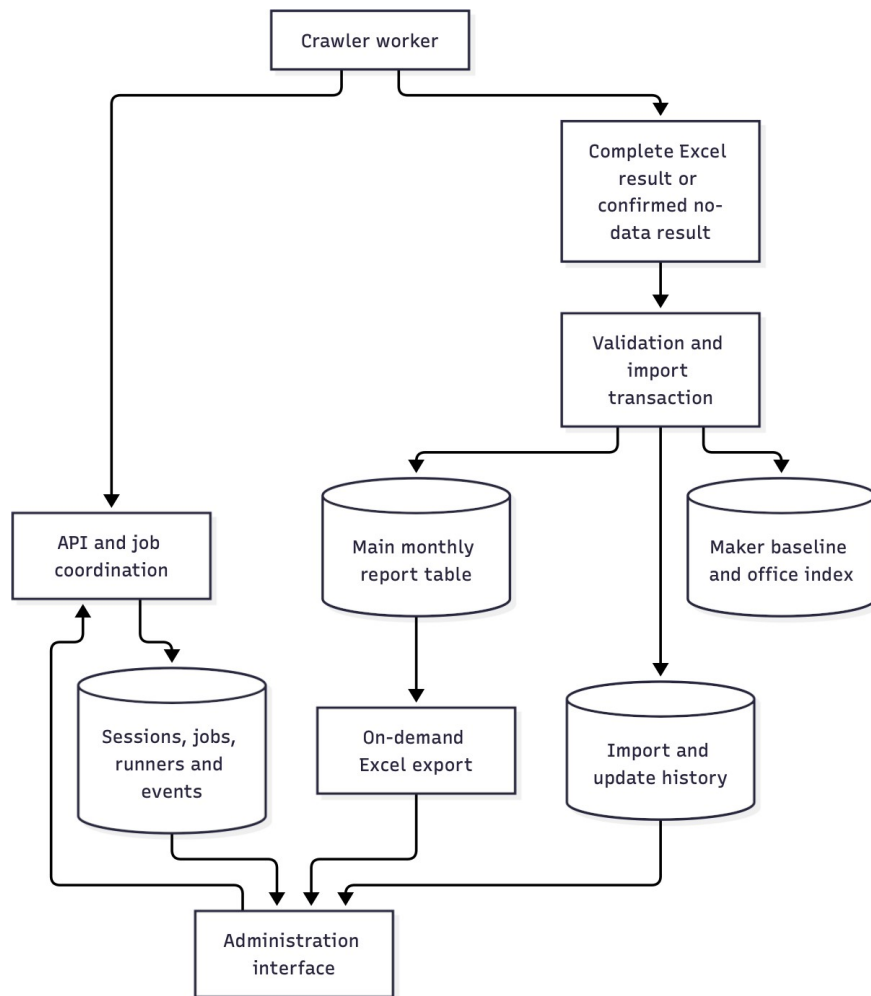


Figure 6 in Appendix A shows how operational records and report values meet at that commit boundary.

Data group	Current function	Business result
Users and sessions	User accounts and access sessions control who can administer the system. Report sessions group cases and preserve their run history.	Access and activity can be traced to a run.
Jobs and workers	Each job records its filters, assigned runner, state changes, timestamps, and errors. Each connected runner has at most one active job.	Operators can see progress and resume or retry the correct case.
Main report data	One logical record identifies a filter scope, year, State, RTO, and Maker. January–December values and per-month source information are stored together.	A blank source value remains different from a reported zero.
Maker change data	All-State Maker snapshots, the Maker–State–RTO index, and update runs/tasks support baseline comparison, office discovery, and focused refreshes.	Changed Makers can be investigated without rerunning every normal case.
History and artifacts	Import history records added, unchanged, replaced, or conflicting values. Diagnostic artifacts and encrypted browser state have separate storage paths.	Source results and follow-up actions remain auditable.

The successful source workbook is parsed into SQL; the current design does not rely on a second master Excel file. Ordinary imports do not silently replace conflicting non-empty monthly values. A focused Maker update may replace values only after its Maker, State, RTO, and discovered totals have been validated.

8. CAPTCHA handling and verification

The current crawler captures the challenge shown by VAHAN, and the API associates its image and identifier with the active job. The administration interface can display the image for a six-character operator response or request a refresh. The runner also contains a local recognition attempt, but a separately deployed CAPTCHA service is not part of the current release.

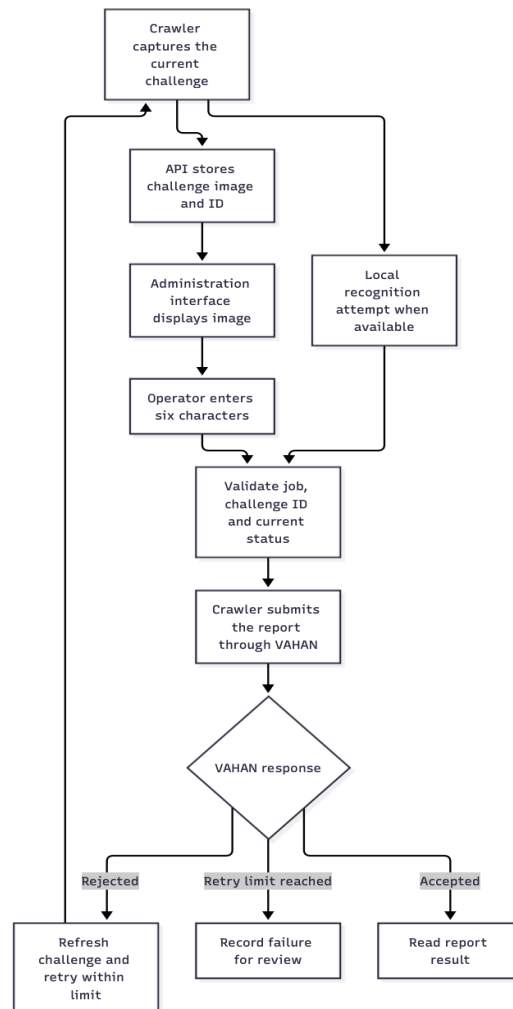


Figure 7 shows the current hand-off.

Function	Current behavior	Result or control
Capture and display	The worker sends the current challenge to the API, which records the image and publishes it to the active dashboard job.	The operator sees the challenge for the correct case.
Receive a response	The API checks the job status and challenge identifier before forwarding the entered value to the assigned worker.	A stale or changed challenge is rejected.
Submit to VAHAN	Before Apply, the worker verifies the filled filters again; VAHAN then decides whether the challenge response is accepted.	A CAPTCHA response alone does not mark a report successful.
Refresh or fail	An invalid response produces a new challenge. After three rejected submissions, the job is marked failed rather than classified as no data.	Verification errors remain visible for retry or review.

In the proposed multi-worker design, a dedicated CAPTCHA service may return a candidate response to the requesting worker. The worker remains responsible for associating the response with the current job and challenge, and for checking VAHAN's actual result. Service availability and acceptance rates still require operational validation.

9. Crawler worker execution

A current Playwright worker owns one browser context and processes one assigned job at a time. The API assigns only an available runner; the worker opens the official VAHAN report, fills the selected filters, and reports state transitions back to the dashboard.

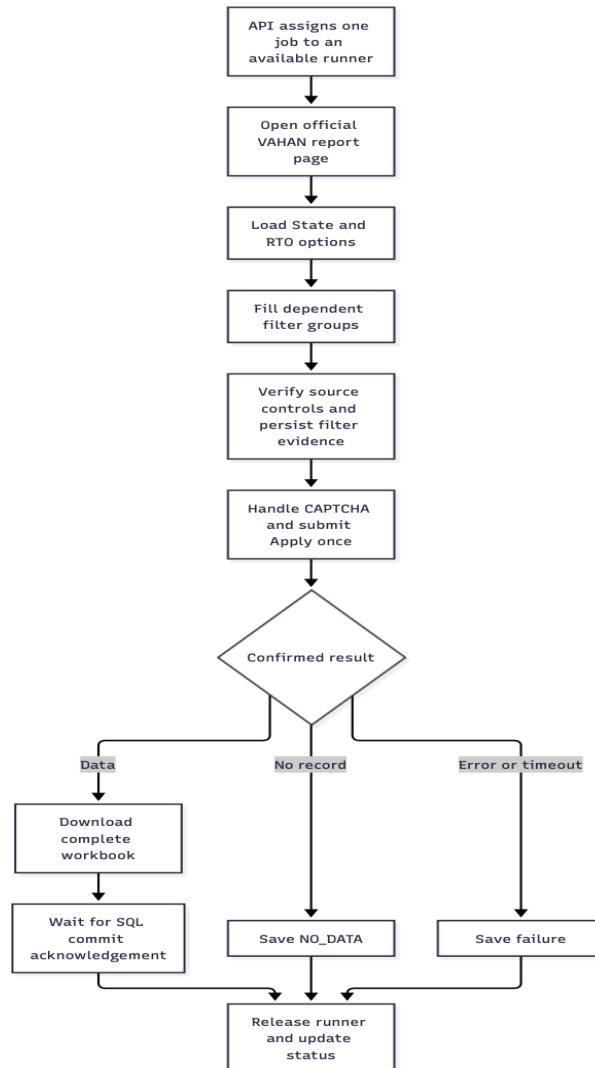


Figure 8 details the current case flow.

Stage	Crawler function	Completion condition
Open and prepare	Open the official report page, restore the browser session, and load the required State and RTO options.	The expected report controls are available; authentication or page-navigation problems are explicit errors.
Fill filters	Fill dependent filter groups, including time, geography, vehicle classification, and report axes. Recheck values after dynamic options load.	The API has accepted a record of expected and observed filter values.
Submit report	Handle the current CAPTCHA, persist the submitting state, click Apply once, and wait for a confirmed report result.	Result reading resumes after late navigation within the same deadline, without submitting the form again.
Finish case	On data, download the full workbook and wait for its SQL save acknowledgement. On confirmed no data, save the no-data result. On error, retain the failure.	The runner is released only after the result has reached a terminal state.

The existing runner registry can identify multiple connected runners. Monthly scheduling, queue distribution, and concurrent workers in Figure 1 are planned extensions; their throughput should be measured after implementation.

10. Excel processing and report aggregation

When VAHAN returns data, the crawler downloads the complete Excel workbook. The API currently performs workbook ingestion and SQL aggregation; there is no independently deployed Excel aggregation worker.

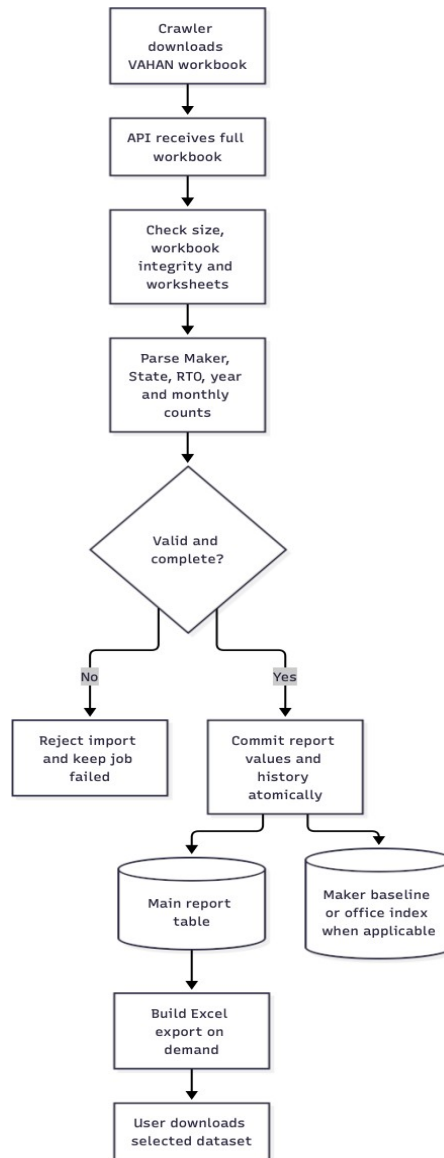


Figure 9 shows the existing processing path from source workbook to the user's export.

Stage	Current function	Data quality result
Receive source	Accept the completed workbook from the assigned crawler, subject to upload size and workbook integrity checks.	Incomplete or damaged exports are rejected.
Read all sheets	Extract rows across every worksheet, identify Maker and month columns, and check the requested year, State, and RTO context.	A partial or ambiguous source does not enter the main table.
Normalize values	Store OTHERS as the Maker label OTHERS; distinguish blank from zero; reject invalid counts and contradictory duplicate rows.	The source meaning is retained in monthly SQL data.

Stage	Current function	Data quality result
Commit results	Write main report values, source references, job completion, and import history in one transaction. Maker baseline and office discovery use their dedicated tables.	The dashboard reports completion only after the save succeeds.
Serve exports	Query the selected dataset and year from the main report table and generate a new Excel workbook when the user requests it.	Exports reflect the saved SQL values and selected scope.

For a future queue-based deployment, Excel parsing can be moved into a dedicated processing component after the crawler downloads a file. It must retain the same full-workbook validation and atomic SQL commit boundary. A merged export remains an on-demand view of committed data, rather than a substitute for the database.